

Bucket 4 Sizing Method

Inverse-decay pairs · How we decide how big each position should be
Simple guide with a worked example · Production as of 11 July 2026

Bucket 4 shorts inverse ETFs (products that lose value when a stock or sector rises) and hedges them by also shorting the underlying. The goal is to harvest structural decay while keeping relative crash risk under control and deploying the sleeve's target budget.

One rule to remember: ρ sets relative crash risk across names. With `scale_to_budget` on, the sleeve still fills its ~3% target — effective per-name crash loss $\approx \rho \times \text{scale_mult}$.

1. The big picture (four steps)

Sizing runs in a fixed order. Crash capping sets relative risk; scale-to-budget then refills the sleeve pro-rata so the YAML target actually deploys; smoothing damps week-to-week churn without undoing true own-risk cuts.

Step 1 — Score the book

Rank pairs by expected decay after borrow costs (continuous high-borrow ramp). Convert scores into relative weights that sum to ~100%.

Step 2 — Crash-cap, then scale to budget

Trim each name so its conditional crash loss $\leq \rho$ of the sleeve, then (with `scale_to_budget: true`) scale the whole capped book pro-rata up to the sleeve target. Riskier names stay smaller than quieter ones.

Step 3 — Post-cap weight smoothing

Trim-only EMA on the final capped weights: raises and new entries fade in; dilution from new entrants fades out; dropped names soft-exit; true own-risk cuts still land immediately.

Step 4 — Leg split + ratchet

Turn each pair's gross into an inverse-ETF short and an underlying short using hedge ratio h . Grow-only inverse ratchet guards hard-to-relocate inventory.

2. Sleeve budget

Bucket 4 gets a fixed share of book target gross (config: `inverse_decay_bucket4.target_weight`, typically 3%). That target is the **deploy** target — after crash trim + `scale_to_budget`, sleeve gross aims at it. Temporary under-deployment (while new names ramp in) stays in cash and self-corrects.

Example sleeve used below: **\$115,500** (illustrative, shaped like a live July 2026 run).

3. Step 1 — Decay / borrow weights

Crash risk is **not** in this score. The score only answers: “How attractive is the decay after expected borrow?”

$$\text{base_score} = \text{decay} / (1 + \text{quad} \cdot \text{borrow}^2) / (1 + \text{linear} \cdot \text{borrow}) / (1 + \text{unc} \cdot \text{borrow_var})$$

High-borrow names are faded continuously (not a binary cliff): score $\times 1.0$ below `borrow_ramp_lo` (80%), linear down to 0 at `borrow_ramp_hi` (120%), using the same posterior borrow estimate as the aversion terms.

Then $\text{weights} = \text{base_score} / \text{sum}(\text{scores})$, clipped to a min/max per name (default 0.5%–35%), then lightly tilted for covariance concentration.

Production knobs: $\text{borrow_linear_aversion} = 1.5$, $\text{borrow_uncertainty_penalty} = 3.0$, $\text{decay_borrow_quad} = 0$, $\text{borrow_ramp_lo/hi} = 0.80 / 1.20$.

4. Step 2 — Conditional crash budget + scale

For each underlying, as of the run date:

```
runup = max(0, price / 252-day median - 1)
retrace =  $\theta \cdot \text{runup} / (1 + \text{runup})$  with  $\theta = 0.5$ 
tail = worst ~3y 20-day drop +  $0.45 \cdot \text{downside vol}$ 
C = max(tail, retrace) conditional crash size
```

Intuition: a name that has already run up a lot can still fall a lot from here, even if recent realized crashes look quiet. We take the larger of historical tail and a fraction of the run-up.

Pair loss per \$1 of gross (L):

```
 $L = (1 - h) \cdot \beta / (1 + h \cdot \beta) \cdot C \cdot (1 + \phi \cdot C)$  with  $\phi = 0.5$ 
```

L is EMA-smoothed across runs (asymmetric): risk-up binds immediately; risk-down fades at $\text{l_ema_alpha} = 0.4$ so a single rolling-window print does not step the whole book.

Per-name trim, then sleeve refill:

```
cap_usd =  $p \cdot \text{sleeve\_budget} / \max(L, L\_floor)$ 
gross_trim = min(solved_gross, cap_usd)
scale_mult = sleeve_budget / sum(gross_trim) (if scale_to_budget)
gross_scaled = gross_trim  $\times$  scale_mult
 $p = 0.75\%$ ,  $L\_floor = 2\%$ 
```

p is a **relative** crash-risk scale. After the pro-rata refill, every full-size name still contributes roughly the same worst-case dollar loss, but that loss is $\approx p \times \text{scale_mult}$ of the sleeve (logged each run as rho_effective). We do **not** hand freed dollars preferentially to uncapped names — the refill preserves the post-cap risk ordering.

5. Step 3 — Post-cap dilution-aware smoothing

Smoothing runs **after** crash-cap + scale so the refill's cross-coupling (one name's cap change re-pricing every other name) is damped too. State lives in `data/b4_weight_ema_state.json` (stage-tagged, with pre-scale own-risk for hard-cut detection).

```
raises → EMA toward target at alpha (0.50)
new entries → ramp at entry_alpha (0.25)
dilution cuts → fade at dilution_alpha (0.25) when scale compresses incumbents but
their own pre-scale capacity is unchanged
soft exits → fade dropped names at soft_exit_alpha (0.35)
hard cuts → immediate if own pre-scale capacity drops > hard_cut_rel (10%)
no-trade band → hold if move < 15% rel or 25 bp abs
```

Deliberately not renormalized inside the smoother: renormalizing would push cut names back above their solved weight. Downstream normalizes. Under-deployment while entries ramp stays in cash.

6. Step 4 — Split gross into two legs + ratchet

```
gross = smoothed_weight × sleeve_budget  
inv_short = gross / (1 + h · β)  
und_short = h · β · inv_short
```

h (hedge ratio) comes from the TR/VCR engine (v7 + v9 tilt): typically between 0.30 and 0.80, centered near 0.45. Higher h = more underlying hedge = less crash beta, but more borrow/margin on the stock leg.

The inverse-ETF short is grow-only by default: we floor it at $\max(\text{solved}, \text{currently held}, \text{persisted floor})$ so we do not cover inventory that is hard to re-borrow. When held size has crept above the fresh target, a continuous trim gradually closes the gap (paired with the underlying so h stays consistent).

7. Worked example — DAMD / AMD

Numbers below are stylized from the 10 July 2026 live shape so the arithmetic is easy to follow. Sleeve budget = **\$115,500**. Pair = short DAMD ($-2 \times$ AMD) + short AMD. Assume the rest of the book is also crash-trimmed so the sleeve-wide scale_mult $\approx$ **6.47**.

Given inputs

Input	Value	Meaning
Sleeve budget	\$115,500	B4 deploy target
Opt2 weight (Step 1)	9.4%	Decay/borrow share of sleeve
AMD conditional crash C	51%	max(tail, retrace)
Hedge ratio h / beta β	0.45 / 2.0	Underlying hedge intensity
Pair loss L	0.377	\$ lost per \$1 gross if C hits
ρ / scale_to_budget	0.75% / true	Relative risk + refill
Book scale_mult	6.47	budget / sum(post-cap)

Step 1 — Solved gross (before crash cap)

$\text{solved_gross} = 9.4\% \times \$115,500 \approx$ **\$10,860**

Step 2a — Per-name crash trim

Allowed crash loss (pre-scale) = $\rho \cdot \text{budget} = 0.0075 \times \$115,500 =$ **\$866**

$\text{cap_usd} = \$866 / 0.377 \approx$ **\$2,297**

$\text{gross_trim} = \min(\$10,860, \$2,297) =$ **\$2,297** → cut to ~21% of the opt2 size

Step 2b — Scale to budget (sleeve refill)

$\text{scale_mult} \approx 6.47$ (whole capped book was ~15% of the \$115,500 target)

$\text{gross_scaled} = \$2,297 \times 6.47 \approx$ **\$14,851**

$\rho_{\text{effective}} \approx \rho \times \text{scale_mult} = 0.75\% \times 6.47 \approx$ **4.85%** of the sleeve if AMD crashes 51%. Relative ordering is preserved: lower-L names still get more gross than higher-L names.

Weight after scale $\approx \$14,851 / \$115,500 \approx$ **12.9%** (up from the 9.4% opt2 weight because DAMD has the lowest L in the book).

Step 3 — Smoothing (steady-state here)

On a quiet week with no new entries / exits, smoothed $\approx$ capped (~\$14,851). If a new name entered, DAMD's scale compression would fade at dilution_alpha = 0.25 instead of jumping; a true own-risk cut (pre-scale capacity down > 10%) would still land immediately.

Step 4 — Two short legs

$\text{inv_short (DAMD)} = \$14,851 / (1 + 0.45 \cdot 2.0) = \$14,851 / 1.90 \approx$ **\$7,816**

$\text{und_short (AMD)} = 0.45 \cdot 2.0 \cdot \$7,816 \approx$ **\$7,034**

Check: $\$7,816 + \$7,034 = \$14,850 \approx$ gross. Check crash loss: $0.377 \times \$14,851 \approx \$5,600 \approx 4.85\%$ of the sleeve.

Example summary

Quantity	Opt2 solved	After trim	After scale
Pair gross	\$10,860	\$2,297	\$14,851
Sleeve weight	9.4%	2.0%	12.9%
Loss if AMD -51%	~\$4,090	\$866 (0.75%)	~\$5,600 (4.85%)

8. Why this design

- Decay score picks who deserves capital; crash budget sets relative risk; scale_to_budget fills the deploy target without breaking that ordering.
- Cap is per-name and dollar-interpretable before scale; after scale, $\rho_{\text{effective}} \approx p \times \text{scale_mult}$ is the number that actually runs.
- Post-cap smoothing stops entry/exit churn and scale-dilution jumps from looking like overnight risk cuts — without delaying true capacity drops.
- Hedge ratio h enters L , so a tightly hedged pair is allowed more gross than a loose one with the same crash C .

9. Where it lives in code / config

- Weights: `scripts/v6_b4_pf_weights.py`
- Crash cap + scale + L EMA: `scripts/b4_crash_budget.py`
- Post-cap smooth + legs + ratchet: `scripts/bucket4_weekly_opt2.py`
- Walk-forward API: `scripts/bucket4_backtest_api.py`
- Hedge/cadence: `scripts/bucket4_hedge_cadence.py`
- Knobs: `config/strategy_config.yml` →
`inverse_decay_bucket4.rules.bucket4_weekly_opt2`
`(crash_budget.*, weight_smoothing.*)`
- Telemetry: `data/runs/<date>/b4_crash_budget.csv, b4_sizing_waterfall.csv`

Production reference · Is-algo · Bucket 4 sizing · 11 July 2026